

M.Tech Thesis Proposal Overview

By

Name: AKASH KAUSHAL

Enroll no.: MT25MCS016

Title: Constraint-Driven Automated Test Vector Generation for Avionics Bus Stimulation and LRU Validation

Domain: Embedded Systems & Formal Software Verification (DO-178C)

The Problem:

In safety-critical avionics, **Line Replaceable Units (LRUs)**—such as Flight Control Computers and Navigation Units—rely on deterministic communication buses like **ARINC 429** and **MIL-STD-1553**. Validating the software parsers that process these bus streams faces significant industry bottlenecks:

1. **The Manual Testing Bottleneck:** In Software-in-the-Loop (SIL) and Hardware-in-the-Loop (HIL) setups, test cases are primarily authored manually or generated via basic parameter sweeps. Manual testing is constrained by human intuition and cannot scale to explore the millions of possible malformed byte combinations.
2. **Complex Edge-Case Vulnerabilities:** Standard testing frequently misses deep combinatorial protocol flaws—such as simultaneous parity bit corruption, out-of-order sub-address sequences, payload-header length mismatches, and burst-rate timing jitter.
3. **Safety & Recertification Risk:** If an LRU bus parser misinterprets a malformed stream, it can lead to memory buffer overflows, state-machine deadlocks, or task starvation. Discovering these logical defects late during iron-bird integration or flight trials leads to major safety risks and expensive recertification cycles.

What I Am Going to Do:

I will develop an automated test generation framework that uses **SMT Solvers (Z3)** and **Coverage-Guided Fuzzing** to mathematically synthesize boundary-condition and malformed test vectors to rigorously stress-test LRU bus parsers.

1. **Protocol Modeling via SMT (Z3):** Model the bit-level structure, parity rules, and valid state transitions of standard avionics protocols (starting with ARINC 429 32-bit words and MIL-STD-1553 command/data frames) as first-order logic constraints using the Z3 SMT solver in Python.
2. **Automated Edge-Case Synthesis:** Formulate mathematical assertions to automatically solve for deterministic, malformed test vectors that force execution down rarely reached error-handling paths:
 - a. *Bit anomalies:* Intentional parity mismatches, invalid Sign/Status Matrix (SSM) configurations, and reserved label injections.
 - b. *Payload boundaries:* Value saturation, scale-factor zero states, and truncated data buffers.

- c. *Sequence anomalies*: Out-of-order commands, missing status words, and state-machine deadlock triggers.
- 3. **SIL Test Harness & Execution**: Build an automated Software-in-the-Loop (SIL) test harness in a Linux environment that streams generated binary vectors directly into a compiled C-based LRU parser via virtual sockets or pipes.
- 4. **Benchmarking & Evaluation**: Quantify code coverage improvements (branch/decision coverage) using gcov/lcov compared to baseline manual and random test generation, while evaluating the parser's error-recovery latency and memory bounds under high-throughput packet bursts.

Expected Deliverable:

A working, Python/Z3-based automated test synthesis tool, an empirical benchmark comparing automated vs. manual test generation, and an M.Tech research paper targeting embedded systems and avionics verification conferences.